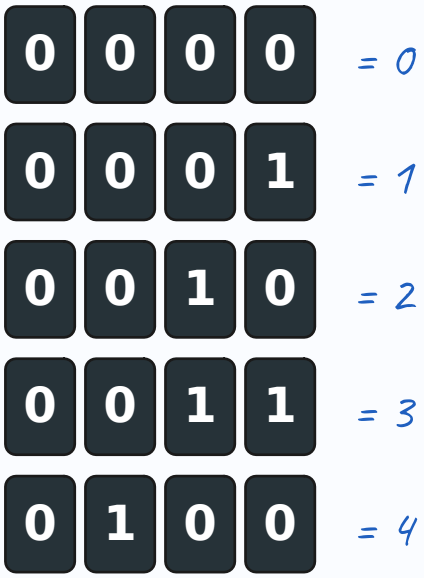
Chapter 10

Number bases

counting with fewer (or more) than ten digits



a mileage counter whose wheels only have 0 and 1 on them: this is binary, the way every computer counts

We count in tens because we have ten fingers. Nothing else about ten is special. A computer's switches have only two states (off/on), so computers count in **twos**: binary. Programmers also use **sixteens** (hexadecimal) because it's a compact way to write binary.

The good news: every base works exactly like the place value you already know from Chapter 1. Only the size of the bundles changes.

Not a lecture topic, but essential for Computer Science modules.

101 Our usual numbers, taken apart column by column

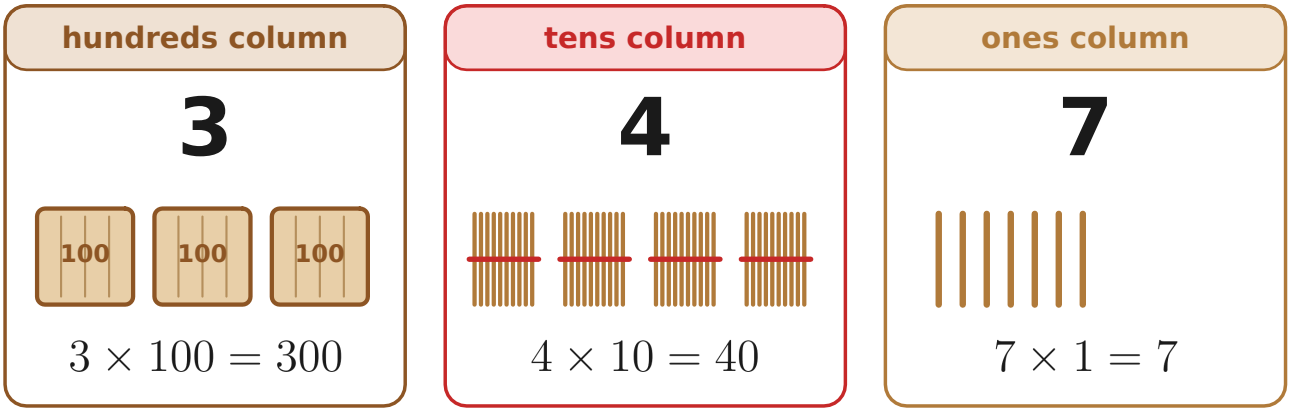
RULE: place value in any base

In base b there are b digits: $0, 1, \ldots, b-1$.

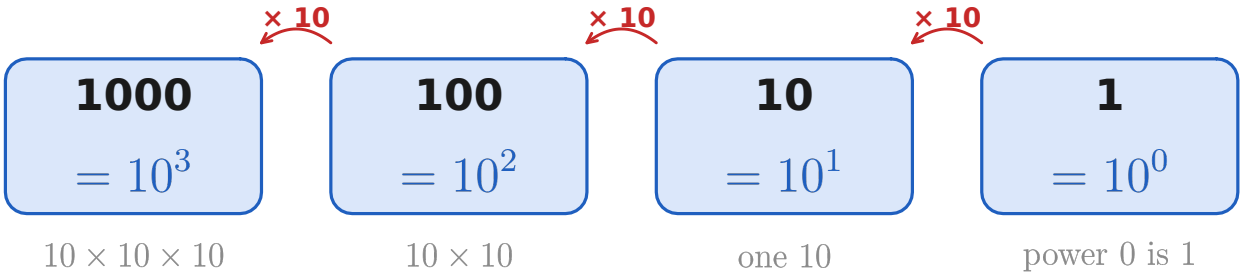
Columns, from the right, are worth $b^0 = 1,\ b^1,\ b^2,\ b^3, \ldots$: each column is worth b times the one to its right.

The value of a number is the sum of (digit × column value). A subscript gives the base: 347_{10}, 10110_2, $2F_{16}$.

Take the number 347. It has three digits, and each digit sits in its own slot, called a **column** (like the columns of a table). Reading from the right: the **ones** column, the **tens** column, the **hundreds** column. The 3, 4 and 7 are not worked out from anything: they are simply the digits of 347, read left to right. What the **column** changes is how much each digit is worth (Chapter 1's crates, bundles and sticks):



The **column value** is what **one** of something is worth in that column: 1 in the ones column, 10 in the tens column, 100 in the hundreds column. Each step to the left is **ten times** bigger, because ten sticks make a bundle and ten bundles make a crate. Now write those column values with powers (Chapter 9). 10^2 just means two tens multiplied:



Put it together. Write the column values as powers over the digits, then multiply each digit by its column value and add:

$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
3	**4**	**7**

$$3 \times 100 + 4 \times 10 + 7 \times 1 = 347$$

The number 10 is called the **base**. A base-10 system needs exactly ten digits (0 to 9) because when a column reaches ten it **rolls over** to 0 and pushes a 1 into the next column, like the mileage counter.

Change the base, change the bundles. In base 2, bundles are made of 2; in base 16, of 16. Everything else stays the same.

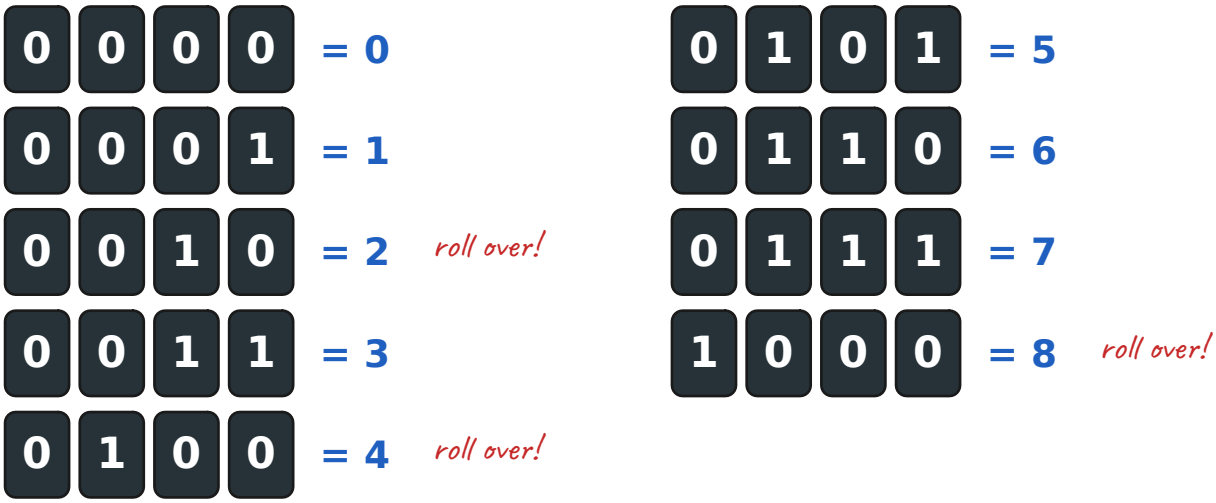
101 Counting in binary: a counter with only 0 and 1

RULE: binary (base 2)

Two digits only: 0 and 1 (called **bits**).

Column values from the right: $1, 2, 4, 8, 16, 32, 64, 128, \ldots$ (the powers of 2).

Each wheel has just two digits. Add 1 and the right wheel goes 0 → 1. Add 1 again: it **rolls over** to 0 and pushes the next wheel on. Watch:



Binary has two digits, 0 and 1, called **bits** (binary digits). The columns are worth 1, 2, 4, 8, 16, ...: each is **double** the one to its right, because two of one column make one of the next.

101 Binary → ordinary: add up the column values

PROCEDURE: base b → decimal

1. Write the column values $1, b, b^2, \ldots$ over the digits, starting from the **right**.
2. Multiply each digit by its column value (in binary: keep the columns with a 1).
3. Add the results.

To read 10110_2 (the little 2 means "in base 2"), write the column values over the digits and add up the ones that have a 1 under them:

16	8	4	2	1
1	**0**	**1**	**1**	**0**

$$16 + 4 + 2 = 22$$

So $10110_2 = 22_{10}$. Read aloud as "one zero one one zero", never "ten thousand one hundred and ten" (that's a base-10 number).

101 Ordinary → binary: making change with coins

RULE: decimal → binary (two methods)

Biggest power first: list the powers of 2 up to the number. Working from the biggest, write 1 and subtract if it fits, 0 if it doesn't, until nothing is left.

Repeated halving: divide by 2, write the remainder (0 or 1), and repeat on the quotient until it is 0. Read the remainders from the **bottom up**.

Imagine a country with coins worth **16, 8, 4, 2 and 1**, and you may use **at most one** of each. Pay 22 using the biggest coins first:

16	8	4	2	1
1	**0**	**1**	**1**	**0**
22: use the 16 → 6 left	*6: 8 is too big → skip*	*6: use the 4 → 2 left*	*2: use the 2 → 0 left*	*0: nothing left → skip*

Used = 1, skipped = 0. Reading the row: **10110**. That's 22 in binary.

Method 2: keep halving (handy for big numbers). Divide by 2 and write down each remainder (Chapter 1); then read the remainders **from the bottom up**:

22 ÷ 2 = 11	**remainder 0**
11 ÷ 2 = 5	**remainder 1**
5 ÷ 2 = 2	**remainder 1**
2 ÷ 2 = 1	**remainder 0**
1 ÷ 2 = 0	**remainder 1**

read upwards:
1 0 1 1 0

101 Base 16 (hexadecimal): running out of digits

RULE: hexadecimal (base 16) and octal (base 8)

Hex digits: 0 to 9, then **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**. Columns: 1, 16, 256, 4096 (powers of 16).

Octal digits: 0 to 7. Columns: 1, 8, 64, 512 (powers of 8).

In base 16 a column can hold up to fifteen before rolling over. But we only have ten digit symbols! So we borrow letters: **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**.

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Columns are worth 1, 16, 256, ... (powers of 16). So $2F_{16} = 2 \times 16 + 15 = 47$.

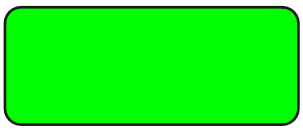
Where you've seen hex: colour codes. A web colour like #FF8800 is three hex pairs: how much **red**, **green** and **blue** light, each from 00 (none) to FF (full):



#FF8800
R255 G136 B0



#1F5FBF
R31 G95 B191



#00FF00
R0 G255 B0



#808080
R128 G128 B128

FF = 15 × 16 + 15 = 255, the biggest two hex digits can hold. #FF8800 is full red, a bit over half green, no blue: orange.

Why programmers love hex: each hex digit is exactly 4 bits, so binary converts in chunks with no arithmetic:

1111	1000	1000	0000
↓	↓	↓	↓
F	**8**	**8**	**0**

11111000 10000000
= F8 80

Octal (base 8, digits 0-7, columns 1, 8, 64) works the same way with 3-bit chunks; you'll see it in Linux file permissions like chmod 755.

101 Decimal → any base: divide and read upwards

PROCEDURE: decimal → base b

1. Divide the number by b. Write down the whole-number quotient and the **remainder** (a remainder is always between 0 and $b-1$).
2. Divide the quotient by b again, and keep going until the quotient is **0**.
3. Write each remainder as a single base-b digit (in hex, 10 to 15 become A to F).
4. Read the remainders from the **bottom up**: the last remainder is the leftmost digit.
5. Check by converting back: digit × column value, added up.

This is the halving method from binary with 2 replaced by b. It works because each division peels off the rightmost digit: the remainder is what doesn't fill a whole bundle of b.

Example: write 200 in hexadecimal.

200 ÷ 16 = 12 **remainder 8**
12 ÷ 16 = 0 **remainder 12 (= C)**

read upwards: C 8

$200 = C8_{16}$ ← check: 12 × 16 + 8 = 192 + 8 = 200 ✓

Example: write 100 in octal (base 8).

100 ÷ 8 = 12 **remainder 4**
12 ÷ 8 = 1 **remainder 4**
1 ÷ 8 = 0 **remainder 1**

read upwards: 1 4 4

$100 = 144_8$ ← check: 1 × 64 + 4 × 8 + 4 = 100 ✓

Example: write 45 in base 3.

45 ÷ 3 = 15 **remainder 0**
15 ÷ 3 = 5 **remainder 0**
5 ÷ 3 = 1 **remainder 2**
1 ÷ 3 = 0 **remainder 1**

read upwards: 1 2 0 0

$45 = 1200_3$ ← check: 1 × 27 + 2 × 9 = 45 ✓

RULE: binary ↔ hex and binary ↔ octal

One hex digit is exactly **4 bits** (because $16 = 2^4$); one octal digit is exactly **3 bits** ($8 = 2^3$).

Binary → hex: split the bits into groups of 4 **from the right**, pad the left group with zeros, and convert each group (0000 = 0 up to 1111 = F). For octal use groups of 3.

Hex → binary: replace each hex digit by its 4 bits (keep leading zeros inside the number).

Example: write 1101011_2 in hex and in octal.

$$\underbrace{0110}_{6}\ \underbrace{1011}_{B} \Rightarrow 6B_{16}$$

← groups of 4 from the right; pad with a 0

$$\underbrace{001}_{1}\ \underbrace{101}_{5}\ \underbrace{011}_{3} \Rightarrow 153_8$$

← groups of 3 from the right

Example: write $2F_{16}$ in binary.

$$2 \to 0010, \quad F \to 1111 \Rightarrow 101111_2$$

← drop the leading zeros at the very front

Check: $101111_2 = 32 + 8 + 4 + 2 + 1 = 47$, and $2F_{16} = 47$ ✓.

101 Adding in binary: 1 + 1 = 10

RULE: binary addition

Line the numbers up on the right and add column by column, **right to left**, as in base 10.

$0+0=0$, $0+1=1$, $1+1=10_2$ (write 0, carry 1), $1+1+1=11_2$ (write 1, carry 1).

Check by converting everything to decimal.

There are only four single-digit sums. The last one rolls over, just as 9 + 1 does in base 10:

0 + 0 = 0 **0 + 1 = 1** **1 + 0 = 1** **1 + 1 = 10**

write 0, carry 1

Work out $1011_2 + 110_2$, column by column from the right, exactly like Chapter 1:

```
  1  1  1
     1  0  1  1
+       1  1  0
-----------------
  1  0  0  0  1
```

1s column: 1 + 0 = 1
2s column: 1 + 1 = 10 → write 0, carry 1
4s column: 0 + 1 + 1 (carry) = 10 → write 0, carry 1
8s column: 1 + 1 (carry) = 10 → 0, carry 1

Check in base 10: 11 + 6 = 17, and $10001_2 = 16+1 = 17$ ✓.

Link to Chapter 7, carefully. A column with **two** bits and no carry coming in is simple: digit = A XOR B, carry = A AND B (a **half adder**). But look at the 4s column above: it adds **three** bits, A, B and the carry that came in. Every column after the first can have three inputs, so a chip uses a **full adder**: the digit is 1 when an **odd** number of the three are 1, and the carry out is 1 when **two or more** are 1. Here's every column of the sum:

column	A	B	carry in	digit (odd?)	carry out (2+?)
1s	1	0	0	**1**	**0**
2s	1	1	0	**0**	**1**
4s	0	1	1	**0**	**1**
8s	1	0	1	**0**	**1**
16s	0	0	1	**1**	**0**

Read the green digits from the bottom up: 10001 ✓. Only the 1s column can never receive a carry, so it can be a half adder. Every other column **might** get one (here the 2s column happened not to), so each needs a full adder. In Chapter 7's words, with C the carry in:

digit = A XOR B XOR C **carry out** = (A AND B) OR (C AND (A XOR B))

101 Subtracting in binary: borrowing a 2

RULE: binary subtraction

Work column by column, right to left: $0-0=0$, $1-0=1$, $1-1=0$.

$0-1$ needs a **borrow**: take 1 from the next column to the left. It arrives as **2** (one of the next column is worth two of this one), so the column becomes $10_2-1=1$.

If the next column is also 0, borrow from further left; every 0 passed over becomes 1.

Example: 1101_2-110_2 (that is $13-6$).

	0	**2**	**2**	
	1	1	0	1
−		1	1	0
	0	1	1	1

1s: 1 − 0 = 1
2s: 0 − 1, borrow from the 4s (its 1 becomes 0):
2 − 1 = 1
4s: now 0 − 1, borrow from the 8s (its 1 becomes 0):
2 − 1 = 1
8s: 0 − 0 = 0

So $1101_2-110_2=111_2$. Check in base 10: $13-6=7=4+2+1$ ✓.

Computers don't actually borrow: they add a negative number stored in "two's complement" form. You'll meet that in Computer Science; on paper, borrowing is fine.

101 Bits, bytes and why a kilobyte might be 1024

A group of **8 bits** is a **byte**. With 8 wheels of 0/1 you can make $2^8 = 256$ different patterns (00000000 to 11111111, i.e. 0 to 255). That's why colour values stop at 255.

Computers like powers of 2, and $2^{10} = 1024$ is close to 1000. So memory sizes were traditionally counted in 1024s: 1 KB = 1024 bytes. Newer notation calls that a **kibibyte** (KiB) and keeps kB = 1000 bytes (the metric prefix from Chapter 5).

101 Any base at all

In base b the digits go from 0 to b − 1, and the columns are worth $b^0 = 1$, b^1, b^2, … . For example, base 3 has digits 0, 1, 2 and columns 1, 3, 9, 27:

$$2101_3 = 2 \times 27 + 1 \times 9 + 0 \times 3 + 1 \times 1$$

← column values are powers of 3

$$= 54 + 9 + 0 + 1 = 64$$

Watch out: a digit can never equal the base. "2" can't appear in binary, and "8" can't appear in octal, just as there's no single digit for ten in base 10.

101 Find the unknown: missing digits and missing bases

Some questions give the value and hide a digit, or hide the base itself. Both are solved by writing the place-value sum with a letter in it:

PROCEDURE: missing digit or base

1. Write the number as (digit × column value) added up, using a letter for the unknown digit, or b for an unknown base (columns $1, b, b^2, \ldots$).
2. Set that sum equal to the value you are given, and solve.
3. Check the answer makes sense: a digit must be a whole number from 0 to $b-1$; a base must be a whole number **bigger than every digit** in the number.

Example: $101x_2 = 11_{10}$. Find the digit x.

$$1 \times 8 + 0 \times 4 + 1 \times 2 + x \times 1 = 11$$ ← columns 8, 4, 2, 1

$$10 + x = 11 \;\Rightarrow\; x = 1$$ ← a valid binary digit ✓

Example: in which base is $21 = 7_{10}$?

$$2 \times b + 1 \times 1 = 7$$ ← columns b and 1

$$2b = 6 \;\Rightarrow\; b = 3$$ ← digits 2 and 1 are both less than 3 ✓

Example: in which base is $100 = 49_{10}$?

$$1 \times b^2 = 49 \;\Rightarrow\; b = 7$$ ← a base is positive, so not −7

Example: $3x_{16} = 59_{10}$. Find the hex digit x.

$$3 \times 16 + x = 59 \;\Rightarrow\; x = 11 = B$$ ← 11 is written B in hex

Example: $25_b = 17_{10}$. Find b.

$$2b + 5 = 17 \;\Rightarrow\; b = 6$$ ← 6 is bigger than the digits 2 and 5 ✓

101 Mistakes to avoid

✗ Common mistakes

✗ $200 \rightarrow 8C_{16}$ ✓ $200 = C8_{16}$

read the remainders from the bottom up

✗ $12_{10} = 12_{16}$ ✓ $12_{10} = C_{16}$

10 to 15 are single hex digits A to F

✗ $1_2 + 1_2 = 2_2$ ✓ $1_2 + 1_2 = 10_2$

there is no digit 2 in binary: write 0, carry 1

✗ $1101011_2 \rightarrow 1101\,011$ ✓ $0110\,1011 = 6B_{16}$

group the bits from the right, not the left

✗ $101_2 = 101$ ✓ $101_2 = 4 + 1 = 5$

the columns are worth 4, 2, 1, not 100, 10, 1; say "one zero one"

Rules in this chapter

Learn these. Every worked example in the chapter is one of these rules applied step by step.

Place value in base b	$(d_2\, d_1\, d_0)_b = d_2 b^2 + d_1 b + d_0$ digits 0 to $b-1$; columns $1, b, b^2, \ldots$ from the right
Binary	digits 0, 1; columns 1, 2, 4, 8, 16, 32, ...
Hexadecimal	digits 0-9, A = 10 ... F = 15; columns 1, 16, 256, ...
Octal	digits 0-7; columns 1, 8, 64, ...
Base b → decimal	multiply each digit by its column value and add
Decimal → base b	divide by b repeatedly until the quotient is 0; read the remainders bottom-up
Decimal → binary	or: take the biggest power of 2 that fits, write 1, subtract, repeat
Binary ↔ hex	groups of 4 bits from the right ($16 = 2^4$)
Binary ↔ octal	groups of 3 bits from the right ($8 = 2^3$)
Binary addition	$1 + 1 = 10_2, \quad 1 + 1 + 1 = 11_2$ right to left, carrying
Binary subtraction	$10_2 - 1 = 1$ 0 − 1: borrow from the left; it arrives as 2
Bits and bytes	n bits $\rightarrow 2^n$ values 8 bits = 1 byte = 256 values (0-255)
Missing digit or base	write the place-value sum with a letter; solve; check digits < base

🔢 Counters and coins: your turn

Answers in grey.

Binary to ordinary

1. 1101_2
2. 100000_2
3. 11111111_2

1. 13 2. 32 3. 255

Ordinary to binary

4. 9
5. 25
6. 100

4. 1001 5. 11001 6. 1100100

Hexadecimal

7. $1A_{16}$ in base 10
8. 200 in hex
9. What colour is #0000FF?
10. Write 10110111_2 in hex

7. 26 8. C8 9. pure blue 10. B7

Arithmetic

11. $101_2 + 11_2$
12. $1111_2 + 1_2$
13. How many values can 4 bits store?

11. 1000_2 (5 + 3 = 8) 12. 10000_2 13. $2^4 = 16$

Other bases

14. 122_3 in base 10
15. Why is 128_8 not a valid octal number?

14. 17 15. octal has no digit 8

More conversions

16. 100 in octal
17. $1110_2 - 101_2$
18. $3C_{16}$ in binary
19. 1101011_2 in octal

16. 144_8 (100 ÷ 8 = 12 r 4, 12 ÷ 8 = 1 r 4, 1 ÷ 8 = 0 r 1) 17. 1001_2 (14 − 5 = 9) 18. 111100_2 (0011 1100) 19. 153_8 (001 101 011)

Find the unknown

20. $101x_2 = 11$. Find the digit x.
21. In which base is 21 = 7?
22. In which base is 100 = 36?
23. $3x_{16} = 59$. Find the hex digit x.
24. 25 in base b equals 17. Find b.
25. $1x1_2 = 7$. Find x.

20. x = 1 (8 + 2 + x = 11) 21. base 3 (2b + 1 = 7) 22. base 6 ($b^2 = 36$) 23. x = B (48 + x = 59, x = 11) 24. b = 6 (2b + 5 = 17) 25. x = 1 (4 + 2x + 1 = 7)